

Routing Traces over a Torq-Annotated Weight Atlas

Posit and experimental protocol — Qwen3-1.7B

P. H. C. Garcia Claude (Opus 5)

14 September 2026

1 Posit

1.1 Statement

P1. A trained transformer’s weight tables carry no recoverable signature of training in any coordinate that reads magnitude alone. The significand distribution of a trained tensor is dominated by the container (the float format’s significand window) and by a scale-free magnitude law shared with untrained tensors.

P2. The signature is carried in *arrangement*: which value sits at which address relative to which other. It is therefore recoverable by a coordinate that reads position and movement, and not by one that reads value.

P3. Under a live query, the selection performed by attention is a movement between addresses. Recording that movement — rather than the values it moves — yields a routing structure that is (i) visible in a single forward pass, (ii) distinct per head, and (iii) not present in the weights read statically.

The experiment tests P3 directly and P1–P2 as controls.

1.2 Prior evidence

Three results from the preceding session, on `EleutherAI/pythia-70m`, motivate the posit. They are stated here as prior observations, not as established findings; the present protocol is the first properly controlled test.

1. *Magnitude blindness.* Gate distributions over two weight tables (layer 0 and layer 1 attention output projections, 512×512) were statistically indistinguishable, and the joint distribution equalled the product of the marginals. The same held for `embed_in` against `embed_out` row-paired by token over 4.2×10^6 paired elements.
2. *Arrangement sensitivity.* The same tensor compared against a shuffle of itself — identical multiset, position destroyed — showed between-row spread of turn counts approximately $5\times$ the shuffled envelope, with 147 of 512 rows outside it.
3. *Routing visibility.* An explicit per-head, per-position forward pass of layer-0 attention on a period-6 token sequence separated the eight heads sharply: two heads placed 40.0% and 30.7% of attention mass on displacements $d \equiv 0 \pmod{6}$, while three others concentrated on $d = 1$ with period-6 mass as low as 1.4%.

Observation 3 was obtained from a single input sequence and cannot distinguish a characteristically periodic head from an incidental lock. Removing that ambiguity is a design requirement of the present protocol (§12).

2 Target and its consequences

Configuration read from the model’s own `config.json`, not from memory:

<code>num_hidden_layers</code>	28
<code>hidden_size</code>	2048
<code>num_attention_heads</code>	16
<code>num_key_value_heads</code>	8
<code>head_dim</code>	128
<code>intermediate_size</code>	6144
<code>tie_word_embeddings</code>	true
<code>rope_theta</code>	1 000 000
<code>torch_dtype</code>	bfloat16

Three consequences bind the protocol.

C1 — bf16 collapses the turn coordinate. A normal bfloat16 significand has 8 bits, so the integer count n satisfies $128 \leq n \leq 255$. Since $n < 360$, *every weight has zero full turns* and the `torq` tuple degenerates to $\{\pm 0, n\}$ on all weight tables. This is a fact about the container, not a defect of the annotation, and it must not be papered over. It is also asymmetric in a useful way: runtime activations and attention values computed in float32 carry 24-bit significands, so the turn coordinate is live on polygon 2 and dead on polygon 1. Declaration **D1** (§3) governs whether this asymmetry is accepted or removed.

C2 — QK-norm sits between the tables and the selection. Qwen3 applies RMSNorm to q and k per head before the score is taken. A trace that deposits pre-norm is not recording what the head compared. All captures in §8 are specified post-norm.

C3 — GQA supplies a free control. Sixteen query heads share eight key/value heads, two queries per group. Paired query heads read *identical* keys and values; any difference in their routing therefore originates solely in their own Q projection. This yields a within-model control that requires no second model and no shuffle, and it is promoted to a primary measurement in §11.

3 Declarations required before execution

Three parameters are free. Each is listed with its consequence. None is chosen here.

D1 — radial depth for weight `torq`. Under C1, bf16 weights yield $\{\pm 0, n\}$ with $n \in [128, 255]$. Options:

- (a) *Accept.* Weights carry residual only; turns are structurally zero. Polygon 1 is a single-lap object. No parameter introduced.
- (b) *Declare a depth δ .* Define $n = m \cdot 2^\delta$ for a stated δ , so that n spans $\lfloor n/360 \rfloor$ turns. This introduces a parameter that must then appear in every receipt, and turn counts become a function of δ rather than of the weight.

Recommendation on record: (a), because it introduces nothing and because the polygon-1/polygon-2 asymmetry it produces is itself informative. The decision is yours.

D2 — polygon side assignment. The construction defines each table as an addition to a polygon, so the t -th table is a t -gon. The enumeration order of tables must be declared, since it fixes which table is which polygon. Candidates: depth-first over the folder tree; layer-major then tensor-major; or an explicit ordered list in the manifest. The protocol below writes whatever order is declared into `manifest.json` as a first-class field so that every downstream artefact is reproducible from it.

D3 — token sequence set. Fixed by §12 as a design requirement, but the specific token IDs are yours. The requirement is: at least four periodic sequences with distinct prime periods, plus one aperiodic control of equal length.

4 Stage 0 — environment

```
D:\positronic-qstmrk\atlas\ # existing sibling-session root
Qwen3-1.7B\ # created by this protocol
```

1. Fetch Qwen/Qwen3-1.7B weights (`model.safetensors`) and `config.json`. Copy `config.json` verbatim into the target folder; do not transcribe values.
2. Write `manifest.json` containing: model id, source URL, file SHA-256, retrieval timestamp, the D1/D2/D3 declarations as resolved, and the table enumeration order in full.
3. Every later stage reads its parameters from `manifest.json`. No stage takes a parameter from a command line.

5 Stage 1 — raw dump

Every table is written as CSV, one file per leaf, values only, so the original weights can be opened and checked independently of any annotation.

5.1 Folder tree

```
Qwen3-1.7B/
  config.json
  manifest.json
  raw/
    embed_tokens/weight.csv # 151936 x 2048 (tied w/ lm_head)
    norm/weight.csv # 2048
    layers/
      00/
        input_layernorm/weight.csv # 2048
        post_attention_layernorm/weight.csv # 2048
        self_attn/
          q_norm/weight.csv # 128
          k_norm/weight.csv # 128
          q_proj/head_00/weight.csv .. head_15/ # 16 x (128 x 2048)
          k_proj/kv_head_00/weight.csv .. kv_07/ # 8 x (128 x 2048)
          v_proj/kv_head_00/weight.csv .. kv_07/ # 8 x (128 x 2048)
          o_proj/head_00/weight.csv .. head_15/ # 16 x (2048 x 128)
        mlp/
          gate_proj/weight.csv # 6144 x 2048
          up_proj/weight.csv # 6144 x 2048
          down_proj/weight.csv # 2048 x 6144
      01/ .. 27/ # identical shape
```

Head splitting is mandatory and is the reason the tree branches: q and o split 16 ways, k and v split 8 ways, and the mapping from query head to KV group is $g(h) = \lfloor h/2 \rfloor$, written into the manifest.

5.2 Format

One matrix row per CSV line. First line is a header of column indices; first field of each subsequent line is the row index. Values are written as the exact decimal expansion of the stored bf16 — every bf16 value is a dyadic rational and therefore terminates — so that the file is a lossless representation and not a rounded view.

5.3 Scale

1 720 574 976 parameters across 1 542 CSV files. At the format above, approximately **24 GB** for **raw/**. Provision accordingly; the expansion is intended.

6 Stage 2 — torq annotation

torq/ mirrors the **raw/** tree exactly, file for file. Each cell becomes the pair {value, ($\pm$ turns, residual)}.

6.1 Derivation, integer only

For each stored 16-bit word:

1. $s \leftarrow$ bit 15 (sign; the plane)
2. $E \leftarrow$ bits 14..7 (exponent field, by mask and shift)
3. $f \leftarrow$ bits 6..0 (mantissa field)
4. $n \leftarrow 128 + f$ if $E > 0$, else f (the radial count)
5. $\text{turns} \leftarrow n \div 360$, $\text{residual} \leftarrow n \bmod 360$

No logarithm, no floating-point operation, no comparison against a real constant appears anywhere in this stage. The exponent is read where it is stored. Under D1(a), step 5 yields $\text{turns} = 0$ for every normal weight and this must be asserted in the output, not assumed.

6.2 Cell format

<value>|<sign><turns>:<residual>
e.g. -0.0172119140625|-0:229

Approximately **52 GB** for **torq/**.

6.3 Mandatory self-check

Before the stage is considered complete, sample 10^4 cells at random, reconstruct n from (turns, residual) by $n = 360 \cdot \text{turns} + \text{residual}$, and assert bit-equality against the stored word. Write the pass count to **manifest.json**. A stage that reports success without running this check is void.

7 Stage 3 — polygons

Polygons are constructed *for tracing*, not for inference. Nothing is inferred from them; they are the coordinate against which a runtime trace is checked.

Polygon 1. The weight table currently being matmulled, held as a polygon rather than as a flat table so that the table can be locked in step with the trace. Per D2, table t is a t -gon with vertex step $360/t$ degrees where $t \mid 360$, and with the residual mapped to vertex $\lfloor \text{residual} \cdot t/360 \rfloor$. Tables whose index does not divide 360 are recorded with their non-integer step in the manifest and carried as-is; they are not rounded.

Polygon 2. A distinct object. It receives, through the fiber relation, the cells of polygon 1 that attention *selects* during the matmul. Polygon 2 stores movement, not value: for each deposit it records the source address on polygon 1, the destination address, and the mass that moved.

```
polygons/
  P1/layers/<LL>/self_attn/<proj>/head_<HH>/vertices.csv
  P2/layers/<LL>/head_<HH>/deposits.csv
  fiber_map.json # P1 address -> P2 address, per D2
```

8 Stage 4 — runtime

8.1 Forward pass discipline

The forward pass is computed **the long way**: explicit loops over layer, head and query position. No fused attention kernel, no batched `scaled_dot_product_attention`. The reason is not performance but observability — the trace must be able to lock step with polygon 1 at each individual score, which a fused kernel does not expose.

For each layer $\ell \in [0, 28)$, head $h \in [0, 16)$, query position i :

1. apply `input_layernorm`;
2. project q, k, v ; reshape to heads; k, v from group $g(h) = \lfloor h/2 \rfloor$;
3. apply `q_norm` to q_h and `k_norm` to $k_{g(h)}$ (C2);
4. apply rotary with `rope_theta` = 10^6 over the full head dimension;
5. for each $j \leq i$: $\text{score}_{ij} = \langle q_i, k_j \rangle / \sqrt{128}$; causal mask above the diagonal;
6. softmax over j ; record the full weight row $W_{\ell hi}$.

Note $\sqrt{128}$ is irrational and is the one unavoidable real constant in the architecture. It is declared here and confined to step 5; it enters no other stage.

8.2 Captures

```
runs/<query_id>/
  tokens.csv # token ids, positions
  attention/L<LL>_H<HH>.csv # full S x S weight matrix, float32
  values/L<LL>_H<HH>.csv # post-norm q, k, v per position
  trace/L<LL>_H<HH>.csv # see Stage 5
```

At $S = 256$: 448 attention files per query, 29 360 128 recorded cells.

9 Stage 5 — trace and check

For each (ℓ, h, i) :

1. **Check against polygon 1.** Locate the cells of the q/k tables that produced the score, and record their polygon-1 vertices. This is the lockstep: every score is attributable to a pair of polygon-1 addresses.

2. **Deposit on polygon 2.** Record *movement*, not value:

$$d = i - j, \quad \text{mass} = W_{\ell hij}, \quad \text{src} \rightarrow \text{dst on the fiber map.}$$

3. **Torq the deposit.** Attention weights are float32, so $n \in [2^{23}, 2^{24})$ and the turn coordinate is live. Record $\{\text{mass}, (\pm \text{turns}, \text{residual})\}$ per deposit — this is the only place in the experiment where turns are non-degenerate under D1(a).

trace/ row format:

```
layer,head,kv_group,i,j,d,mass,mass_turns,mass_residual,p1_src_vertex,p1_dst_vertex,p2_vertex
```

10 Stage 6 — output: the 3D map

The deliverable is a node-and-edge map in three dimensions, one per query.

Nodes. (ℓ, h, i) — layer as depth, head as one planar axis, position as the other. $28 \times 16 \times S$ nodes.

Edges. One per deposit above a mass floor (declared in the manifest; suggested floor 0.01, stated so it can be changed and recorded). Edge from (ℓ, h, j) to (ℓ, h, i) .

Arrows. Each edge carries a vector whose direction is the displacement d mapped onto the polygon-2 ring, and whose magnitude is the mass. The arrow is the object of interest: it points at where the value routed.

```
runs/<query_id>/map.json
{ "nodes": [{ "id", "layer", "head", "pos", "kv_group" }],
  "edges": [{ "src", "dst", "d", "mass", "p2_vertex", "vec": [x,y,z] }],
  "meta": { manifest hash, mass floor, S, declarations } }
```

11 Analysis

Four measurements, in order. The first three are controls and must be run before any claim is made about the fourth.

A1 — container control (tests P1). Compare the residual distribution of **torq/** against a random tensor of the same dtype and shape. Expectation: indistinguishable. A difference here means the container model is wrong and everything downstream is suspect.

A2 — arrangement control (tests P2). Compare each weight table against a shuffle of itself, measuring between-row spread of residuals. Expectation, from the pythia prior: trained spread substantially exceeds the shuffled envelope. This is the control that was run too late in the previous session.

A3 — GQA control (tests P3, free from C3). For each KV group, compare the displacement profiles of its two query heads. They read identical keys and values. If paired heads route more similarly than unpaired heads, the routing is partly determined by shared KV; if they route independently, it is determined by Q alone. Both outcomes are informative and neither requires a second model.

A4 — routing (the experiment). Per head, the mass-weighted displacement profile and the sink fraction $W_{\ell_{hi}0}$. A head is *period-locked* if it concentrates mass on $d \equiv 0 \pmod{p}$ for the input’s period p and tracks p when p changes across the input set. A head that always fires at one fixed d regardless of p is positional, not periodic. Distinguishing these is the purpose of §12 and is exactly what the previous session could not do.

12 Input set

Minimum, per D3:

- four periodic token sequences with periods 3, 5, 7, 11;
- one aperiodic sequence of equal length;
- all sequences of the same length S , $S \geq 6 \times \max(p)$ so every period has at least six repetitions.

A head reported as period-locked must lock to 3 on the period-3 input, to 5 on the period-5 input, and so on. Any head locking to a fixed displacement across all five is positional and is reported as such.

13 Stop condition

The protocol terminates and the result is reported — positive or negative — after A1–A4 are complete on the five-input set. Continuation beyond that point is conditional on the arrows in §6 showing structure that A3 and A4 cannot already account for. If the arrows are flat, that is the finding, and it is reported as such rather than extended.

14 Failure conditions

The run is void, not merely negative, if any of the following occurs:

- Stage 2’s self-check is skipped or reports any mismatch.
- Any stage invokes a logarithm, a transcendental, or a real-valued comparison outside the declared $\sqrt{128}$ of §8.
- Any stage takes a parameter not present in `manifest.json`.
- A fused attention kernel is used in Stage 4.
- A claim in §11 is made before its control has run.